

CS 551 Systems Programming

Lecture 6: Slices

Where we are

Lecture 4 told you what a slice *is*: a fat pointer, two machine words, an address plus a length. `sizeof::<&[i32]>()` is 16 and you know why.

This lecture is about what a slice is *for*.

The short version: a slice is a **borrow that names part of something**. Because the compiler tracks it like any other borrow, an entire category of bug stops compiling. We are going to write that bug first, watch it not get caught, and then fix it by changing one type.

Today's path

1. **The bug.** A function that returns a position into a string.
2. **String slices.** `&str`, and the range syntax.
3. **The fix.** The same function, returning a slice, and what the compiler does differently.
4. **Slices of everything.** Arrays, `Vec`, and why your functions should take `&[T]`.
5. **Working with slices.** Patterns, chunks, and windows.

1. The bug

A function that returns a position

Find the first word

We want the index where the first word ends. Reasonable first attempt:

```
1 fn first_word(s: &String) -> usize {
2     let bytes = s.as_bytes();
3
4     for (i, &item) in bytes.iter().enumerate() {
5         if item == b' ' {
6             return i;
7         }
8     }
9
10    s.len();
11 }
```

Before we talk about what it does: **will it compile?**

No, and you already know why

```
error[E0308]: mismatched types
  |
  | fn first_word(s: &String) -> usize {
  |                               ----- expected `usize` because of return type
  |                               s.len();
  |                               - help: remove this semicolon to return this value
```

The semicolon on the last line discards the value, so the body evaluates to `()`.

Same rule as Lecture 3, in a new place. It will keep showing up. Drop the semicolon and it compiles.

Now it works

```
1 fn first_word(s: &String) -> usize {
2     let bytes = s.as_bytes();
3     for (i, &item) in bytes.iter().enumerate() {
4         if item == b' ' {
5             return i;
6         }
7     }
8
9     s.len()
10 }
```

- `as_bytes()` gives us the string's raw bytes as a slice.
- `enumerate()` pairs each item with its index.
- `b' '` is a **byte literal**: the single byte for a space, not a `char`.

It returns 5 for `"hello world"`. Correct, useful, and about to go badly wrong.

LIVE CODING CHALLENGE

Call `first_word(&s)` to get an index, then `s.clear()`, then print the index.

The compiler accepts this. What is the index describing now?

Pause here. Try it before the next slide.

The bug that compiles

```
1 fn main() {  
2     let mut s = String::from("hello world");  
3     let index = first_word(&s);  
4  
5     s.clear();  
6  
7     println!("index = {index}");  
8 }
```

The bug that compiles

```
1 fn main() {  
2     let mut s = String::from("hello world");  
3     let index = first_word(&s);  
4  
5     s.clear();  
6  
7     println!("index = {index}");  
8 }
```

index is 5, and it is just a number

The bug that compiles

```
1 fn main() {  
2     let mut s = String::from("hello world");  
3     let index = first_word(&s);  
4  
5     s.clear();  
6  
7     println!("index = {index}");  
8 }
```

`clear()` empties `s`. Its value is now ""

The bug that compiles

```
1 fn main() {  
2     let mut s = String::from("hello world");  
3     let index = first_word(&s);  
4  
5     s.clear();  
6  
7     println!("index = {index}");  
8 }
```

index is still 5, and it now means nothing

Nothing stopped you

```
index = 5
```

It compiles cleanly. It runs. It prints a number that refers to a position in a string that no longer has any positions.

index and **s** are two separate things, and only one of them changed. The compiler had no reason to object, because nothing in the types said they were related.

That is the problem slices solve: a type that *is* the connection, so the compiler can see it.

Why this is the bad kind of bug

`index` is just a `usize`. It is a number. It has **no connection** to the string it came from.

- The compiler cannot help, because nothing is wrong at the type level. A `usize` is a `usize`.
- Nothing crashes here. It silently means the wrong thing, and the damage shows up later, somewhere else, when someone indexes with it.
- And `clear()` is only the easy example. Push to the string, truncate it, reassign it: the index goes stale every time and never says so.

We have a value that is *supposed* to be tied to another value. The language does not know that. **That is the actual problem, and slices are the actual solution.**

2. String slices

Naming part of a string

A slice is a view into a string

```
1 let s = String::from("hello world");
2 let hello = &s[0..5];
3 let world = &s[6..11];
4 println!("{hello} {world}");
```

```
s: String          "hello world"  <-- heap
+-----+
| ptr  -----|-----+
| len: 11      |
| cap: 11      |
+-----+
world: &str        | No copy! Just points into `s`
+-----+
| ptr  -----|-----+
| len: 5       |
+-----+
```

Range syntax

Form	Means
<code>[start..end]</code>	<code>start</code> through <code>end - 1</code>
<code>[..end]</code>	0 through <code>end - 1</code>
<code>[start..]</code>	<code>start</code> through the end
<code>[..]</code>	everything
<code>[start..=end]</code>	<code>start</code> through <code>end</code> , inclusive

The type of all of these is `&str`.

NB Using `..=` as a *slice bound* costs nothing: the end index is computed once. (Do not confuse this with *iterating* a `..=` range, which does carry a small penalty. That is the performance lecture's problem, not this slide's.)

How to say it out loud

`str` is usually said "**stir**", rhyming with "her", though you will hear it spelled out too. `&str` read aloud is almost always "**string slice**", which is what the standard library calls it.

Written	Said	Is
<code>String</code>	"string"	owned, growable, on the heap
<code>&String</code>	"ref string"	a borrow of one of those
<code>str</code>	"stir"	the unsized type, rarely written bare
<code>&str</code>	"string slice"	a view into UTF-8 bytes someone else owns
<code>&'static str</code>	"static string slice"	a slice of the binary itself, like a literal

Say "string slice" when you mean `&str`. It is the one nobody mishears.

Range syntax example

```
1 let s = String::from("hello world");
2
3 let a = &s[0..5];    // bytes 0 through 4
4 let b = &s[..5];     // same thing; 0 is the default start
5 let c = &s[6..];     // byte 6 through the end
6 let d = &s[..];      // the whole string
7 let e = &s[6..=7];   // bytes 6 and 7, inclusive at the top
8
9 println!("{a}|{b}|{c}|{d}|{e}");
```

`&str` is bytes, not characters

This is the part that surprises people, and it matters for a systems course.

```
1 let s = String::from("héllö");  
2  
3 println!("{}", s.len());           // 6, not 5  
4 println!("{}", s.chars().count()); // 5
```

`é` is two bytes in UTF-8. `len()` is a **byte** count, because that is what the fat pointer carries and what indexing uses.

So what happens if you slice through the middle of a character?

LIVE CODING CHALLENGE

Take `let s = String::from("héllö");` and slice `&s[0..2]`.

Compile error, panic, or the first two characters?

Pause here. Try it before the next slide.

Slicing off a char boundary

```
1 let s = String::from("héllö");  
2 let bad = &s[0..2];  
3 println!("{bad}");
```

```
thread 'main' panicked at src/main.rs:2:16:  
end byte index 2 is not a char boundary; it is inside 'é' (bytes 1..3 of string)
```

It compiles. The check happens at runtime, and it panics rather than handing you invalid UTF-8.

That is the guarantee `&str` carries: the bytes in it are always valid UTF-8. Rust would rather stop your program than let you violate it. When you need to iterate text safely, use `.chars()`, or `char_indices()` if you need positions.

3. The fix

Same function, different return type

first_word, returning a slice

```
1 fn first_word(s: &str) -> &str {
2     let bytes = s.as_bytes();
3
4     for (i, &item) in bytes.iter().enumerate() {
5         if item == b' ' {
6             return &s[0..i];
7         }
8     }
9
10    s
11 }
```

Two changes:

1. It returns `&str` instead of `usize`.
2. It takes `&str` instead of `&String`. More on that shortly.

LIVE CODING CHALLENGE

Same program as before, except `first_word` now returns a `&str` instead of a `usize`. Call it, then `s.clear()`, then print the result.

What changed?

Pause here. Try it before the next slide.

The same bug, now rejected

```
1 fn main() {  
2     let mut s = String::from("hello world");  
3  
4     let word = first_word(&s);  
5  
6     s.clear();  
7  
8     println!("{word}");  
9 }
```

The same bug, now rejected

```
1 fn main() {  
2     let mut s = String::from("hello world");  
3  
4     let word = first_word(&s);  
5  
6     s.clear();  
7  
8     println!("{word}");  
9 }
```

first_word hands back a slice, which is an **immutable borrow** pointing into **s**

The same bug, now rejected

```
1 fn main() {  
2     let mut s = String::from("hello world");  
3  
4     let word = first_word(&s);  
5  
6     s.clear();  
7  
8     println!("{word}");  
9 }
```

`clear()` needs a **mutable** borrow, but `word` is still holding an immutable one

Reference rules. One mutable reference, *or* any number of immutable ones. Never both, and this is both.

The same bug, now rejected

```
1 fn main() {  
2     let mut s = String::from("hello world");  
3  
4     let word = first_word(&s);  
5  
6     s.clear();  
7  
8     println!("{word}");  
9 }
```

What's the compiler say now?

```
error[E0502]: cannot borrow `s` as mutable because it is also borrowed as immutable
|
|   let word = first_word(&s);
|                   -- immutable borrow occurs here
|   s.clear();
|   ^^^^^^^^^ mutable borrow occurs here
|   println!("{word}");
|                   ---- immutable borrow later used here
```

Identical `main`. Identical bug. **It no longer compiles**, and the error names all three lines involved.

What actually changed

`usize` is a number with no relationship to anything.

`&str` is a **borrow**, and the borrow checker already knows what to do with borrows:

- `word` holds a shared borrow of `s`.
- `clear()` needs `&mut self`, an exclusive borrow.
- `word` is used after the `clear()`, so the shared borrow is still live.
- Many readers or one writer, never both. Rejected.

We did not add a rule. We used the existing one, by choosing a return type that carries the relationship the `usize` was throwing away.

This is the lesson of the lecture. Encode the relationship in the type, and the compiler enforces it for free.

4. Slices of everything

Not just strings

Three ways to hold a sequence

`[i32; 3]`
fixed-size array

stack
+-----+
| [1][2][3] |
+-----+

OWNS the data
size in the type
len known at compile

`Vec<i32>`
growable vector

stack heap
+-----+ +-----+
| ptr ----|-->|[1][2] |
| len: 2 | +-----+
| cap: 4 |
+-----+

OWNS the data
len known at runtime

`&[i32]`
slice (a view)

stack
+-----+
| ptr -----|--> (someone
| len: 2 | else's
+-----+ buffer)

OWNS NOTHING
borrows it

What does this mean, practically speaking?

- `[T; N]` owns its elements, and the length is part of the type.
- `Vec<T>` owns a heap buffer it can grow.
- `&[T]` owns nothing. It points at someone else's elements and knows how many.

A slice can borrow from **either** of the other two. That is what makes it useful.

So take `&[T]`, not `&Vec<T>` when you can!

```
1 fn total(xs: &[i32]) -> i32 {
2     xs.iter().sum()
3 }
4
5 fn main() {
6     let array = [1, 2, 3];
7     let vector = vec![4, 5, 6];
8
9     println!("{}", total(&array));           // &[i32; 3] coerces
10    println!("{}", total(&vector));           // &Vec<i32> coerces
11    println!("{}", total(&vector[1..]));      // already a slice
12    println!("{}", total(&[]));               // an empty one
13 }
```

One signature, four callers. The array and the `Vec` both coerce to `&[i32]` automatically.

The narrower version rejects callers

```
1 fn total(xs: &Vec<i32>) -> i32 {
2     xs.iter().sum()
3 }
4
5 fn main() {
6     let array = [1, 2, 3];
7     println!("{}", total(&array));
8 }
```

```
error[E0308]: mismatched types
  |         println!("{}", total(&array));
  |                               ^^^^^^ expected `&Vec<i32>`, found `&[i32; 3]`
```

`&Vec<i32>` accepts exactly one thing. It also implies you might grow the vector, which you cannot do through a shared reference anyway.

Rule of thumb

Take `&[T]` when you need to read a sequence. Take `&Vec<T>` essentially never. The same rule gives you `&str` over `&String`, which is why `first_word` changed its parameter type two slides ago.

5. Working with slices

Slice patterns

Everything you learned about `match` in Lecture 2 works on slices:

```
1 fn describe(v: &[i32]) -> String {
2     match v {
3         [] => "empty".to_string(),
4         [x] => format!("just {x}"),
5         [first, last] => format!("a pair: {first}, {last}"),
6         [first, .., last] => format!("{first} through {last}, {} items", v.len()),
7     }
8 }
9
10 fn main() {
11     println!("{}", describe(&[]));
12     println!("{}", describe(&[7]));
13     println!("{}", describe(&[1, 2]));
14     println!("{}", describe(&[1, 2, 3, 4]));
15 }
```

`..` in a slice pattern

`..` matches **any number of elements**, which is what makes the last arm above work for a slice of any length.

You can bind it, too. `[first, rest @ ..]` gives you the head and a slice of the tail, with no copying and no bounds arithmetic.

Chunks and windows

```
1 let data = [1, 2, 3, 4, 5, 6, 7];
2
3 for c in data.chunks(3) {
4     print!("{c:?} ");           // [1,2,3] [4,5,6] [7]
5 }
6 println!();
7
8 for c in data.chunks_exact(3) {
9     print!("{c:?} ");           // [1,2,3] [4,5,6] (drops the remainder)
10 }
11 println!();
12
13 for w in data.windows(3) {
14     print!("{w:?} ");           // [1,2,3] [2,3,4] [3,4,5] ...
15 }
16 println!();
```

Which of the three to reach for

- `chunks(n)` splits into non-overlapping pieces; the last may be short.
- `chunks_exact(n)` drops a short remainder, and is faster because the length is uniform.
- `windows(n)` slides, overlapping.

None of these copy anything. Each item is a `&[T]` into the original buffer.

Why `chunks_exact` matters to you

In Project 1 you write a binary file whose data section is fixed-width hash records back to back: 16 bytes for md5, 32 for sha256, 64 for sha3-512.

Reading it back is exactly this:

```
1 let data: &[u8] = &[0xde, 0xad, 0xbe, 0xef, 0x01, 0x02, 0x03, 0x04];
2 let hash_width = 4;
3
4 for hash in data.chunks_exact(hash_width) {
5     println!("{hash:02x?}");
6 }
```

No index arithmetic, no off-by-one, no bounds checks you wrote yourself. The slice knows how long it is, and `chunks_exact` divides it up.

Mutable slices

Everything so far has been read-only. `&mut [T]` lets you write:

```
1 let mut v = vec![3, 1, 2];
2
3 v.sort();           // Vec derefs to &mut [i32]
4 println!("{v:?}");  // [1, 2, 3]
5
6 let s: &mut [i32] = &mut v;
7 s[0] = 99;
8 s.swap(0, 2);
9 println!("{v:?}");  // [3, 2, 99]
```

`sort`, `swap`, `reverse`, and `fill` are all slice methods, not `Vec` methods. `Vec` gets them by dereferencing to `&mut [T]`.

And the borrow rules still apply, which is why `split_at_mut` exists. That was Lecture 5's example, and now you know why it returns two slices.

Bounds checks

Indexing a slice is checked at runtime. `v[10]` on a three-element slice panics rather than reading whatever is next in memory.

That check costs something, and the usual way to avoid paying for it is not to index at all:

```
1 let v = vec![1, 2, 3];
2
3 let mut total = 0;
4 for i in 0..v.len() {
5     total += v[i];           // bounds-checked every iteration
6 }
7
8 let total2: i32 = v.iter().sum(); // no indexing, no checks to elide
9 println!("{total} {total2}");
```

The iterator version is idiomatic, and usually faster, because there is no index whose bounds anything has to check. Usually, not always: measure before you lean on it.

Recap

1. **A slice is a borrow that names part of something.** Pointer plus length, owning nothing.
2. **Returning a position throws away the relationship.** Returning a slice keeps it, and the borrow checker enforces it.
3. `&str` is **byte-indexed**, and slicing off a UTF-8 boundary panics. Use `.chars()` for characters.
4. **Range syntax:** `[a..b]`, `[..b]`, `[a..]`, `[..]`, `[a..=b]`.
5. **Take** `&[T]`, not `&Vec<T>`. Take `&str`, not `&String`. One signature, every caller.
6. **Slice patterns** work in `match`, including `..` and `rest @ ..`.
7. `chunks_exact` and `windows` carve a buffer up without copying. You will use the first one in Project 1.
8. **Prefer iterators to indexing.** Fewer bugs, fewer bounds checks.

Exercises

1. Write `fn last_word(s: &str) -> &str`. Then write a `main` that calls it and tries to mutate the string while the result is alive. Read the error.
2. Take the `usize` version of `first_word` and find a *second* way to make its result meaningless, without calling `clear()`.
3. Predict, then check:
 - `let s = String::from("héllö"); let x = &s[0..1];`
 - `let s = String::from("héllö"); let x = &s[0..3];`
 - `let v = vec![1,2,3]; let s = &v[1..5];`

(More) Exercises

4. Write `fn evens(xs: &[i32]) -> Vec<i32>` returning only the even values. Why can it not return `&[i32]`?
5. Use `chunks_exact(2)` to turn `&[1, 2, 3, 4, 5, 6]` into `Vec<(i32, i32)>`. What does the leftover element do?
6. Change `describe` to add an arm for exactly three elements. Where must it go relative to the other arms, and why?

Reading

- **The Rust Programming Language**, chapter 4.3 (The Slice Type) and chapter 8.2 (Storing UTF-8 Encoded Text with Strings).
- **Rust by Example**, the sections on arrays and slices.
- **The `std::slice` docs**. Genuinely worth skimming the method list once; most of what you will want already exists.

Next lecture: structs and methods. Giving your own types names, fields, and behavior.

Next: Lecture 7

Structs and Methods

Your own types, and the data they own.